Saorinians Student Management System

A presentation-ready explanation of the PHP/MySQL application for Santa Clarita International School. This document explains what each code layer does, how data moves through the system, and how the role-based modules support school operations.

1. Project Purpose and Architecture

Saorinians is a server-rendered PHP application backed by MySQL. Users open PHP pages in a browser. Each page loads configuration and shared helpers, checks the session and role, reads or writes data through PDO prepared statements, then renders Bootstrap-based HTML.

â¢ Presentation flow: Browser -> PHP page -> authentication/authorization -> database helper -> MySQL -> HTML response.
â¢ Main entry points: index.php redirects to login or dashboard; login.php authenticates; register.php creates a pending request; dashboard.php selects a workspace by role.
â¢ Shared layers: config/ stores settings, includes/ stores reusable logic and layout, modules/ stores role-specific screens, assets/ stores CSS, JavaScript, and images, uploads/ stores profile images and submitted files.

2. Configuration and Startup Code

â¢ config/app.php defines BASE_URL, application name/version, upload limits, session timeout, pagination, and date formats. It also creates upload directories when needed.
â¢ config/database.php defines MySQL connection values and creates one PDO connection through the Database singleton. Exceptions are enabled, associative fetches are the default, and emulated prepares are disabled.
â¢ index.php loads the app and auth files, checks isLoggedIn(), and redirects to dashboard.php or login.php.

3. Authentication and Security

includes/auth.php provides the security boundary used by almost every protected screen. login() loads a user by username, verifies a password hash with password_verify(), and stores user_id, username, role, name, and login time in the PHP session.

â¢ isLoggedIn() starts the session, rejects missing user IDs, expires inactive sessions after SESSION_TIMEOUT, and refreshes the activity timestamp.
â¢ requireLogin() redirects unauthenticated visitors. requireAdmin(), requireTeacher(), and requireStudent() enforce role-specific access before page logic runs.
â¢ logout() clears session values, expires the session cookie, and destroys the session.
â¢ Registration uses CSRF validation, input sanitization, role allow-listing, password length checks, email validation, duplicate checks, and creates an admin-review request instead of immediately creating an account.
â¢ Passwords are stored as hashes. Database queries use PDO prepared statements with bound values, reducing SQL injection risk.

4. Database Layer and Schema

database.sql creates the student_management database and seeds demo accounts, courses, sections, enrollments, and an announcement. The schema separates identity, academic structure, learning activity, communication, and uploaded resources.

â¢ Identity: users and user_profiles. One user has one profile; roles are admin, teacher, or student.
â¢ Academic structure: courses -> sections -> enrollments. A section connects a course to an optional teacher and enrolled students.